

ARCHITECTURE

Architecture

Overview

BankOne is a staff-facing core banking shell:

- **Backend:** Spring Boot 4.1, Java 21, packaged as WAR, deployed on **Open Liberty**
- **Frontend:** Angular SPA with JWT Bearer auth
- **Database:** PostgreSQL (bankone)

flowchart LR

```
UI[Angular SPA :4200] -->|HTTPS/HTTP JWT| API[Spring Boot on Liberty :9080]
API --> PG[(PostgreSQL :5432)]
```

Package dependencies

flowchart TB

```
subgraph backend [com.bankone]
    auth[auth]
    user[user]
    role[role]
    customer[customer]
    account[account]
    dashboard[dashboard]
    common[common]
end
auth --> user
auth --> role
auth --> common
customer --> account
account --> customer
account --> common
user --> role
user --> common
dashboard --> customer
dashboard --> account
dashboard --> user
```

Empty placeholder packages (no classes yet): transaction, report, audit, beneficiary.

Layering (per feature module)

```
controller → service (interface + impl) → repository → entity
                ↓
                dto / enums / specification / util
```

Cross-cutting concerns live under `com.bankone.common`:

Area	Classes
Security	SecurityConfig, JpaAuditingConfig
Exceptions	GlobalExceptionHandler, domain exceptions

Area	Classes
Audit base	AuditableEntity
Utils	BusinessIdFormatter

Frontend structure

```
src/app/
  core/          # auth, guards, interceptors, services, models, layout, config
  features/      # login, dashboard, customers, accounts, employees, profile, management
  app.routes.ts
```

Protected routes use `authGuard` with optional `data.roles`.

Deployment

Mode	Port	Notes
Open Liberty (primary)	9080 HTTP / 9443 HTTPS	<code>scripts/redeploy-liberty.sh</code>
Embedded Tomcat	8080	<code>application.properties</code> <code>server.port</code>
Angular	4200	Talks to API base in <code>api-config.ts</code>

WAR artifact: `target/bankone-0.0.1-SNAPSHOT.war` → `$WLP_HOME/usr/servers/bankone/apps/`

Security architecture

1. Stateless JWT (`SessionCreationPolicy.STATELESS`)
2. Login public; all other endpoints authenticated
3. Role checks via `SecurityConfig` matchers (+ `@PreAuthorize` on some user APIs)
4. Roles loaded from DB on each request (`CustomUserDetailsService` → `UserRole`), **not** embedded in JWT claims
5. Account lockout via `LoginAttemptService` / user lock fields

Data architecture

- Schema evolution: Hibernate `ddl-auto=update`
- Explicit SQL: `schema.sql` creates `account_ordinal_seq` only
- Soft business IDs: `customerCode` is formatted in JSON (`BusinessIdFormatter`), not a DB column

Related docs

- [CALL_FLOW.md](#)
- [CLASS_DIAGRAM.md](#)
- [DATABASE_SCHEMA.md](#)